COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 6, 2026	**DATE SUBMITTED:**	October 7, 2026
NAME:	Dizor, Elijah Gresola, Aryanna Keith G	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

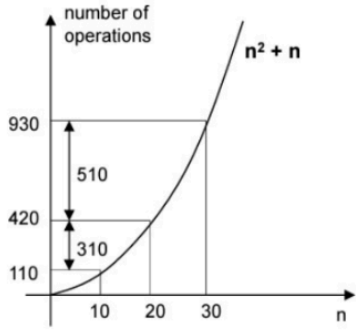
TASK

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Given the above information, the total number of comparisons in the worst case is: **mn comparisons**

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS

The graph represents a quadratic growth rate, O(n^2). As the input size increases, the number of comparisons also increases. When the input size is doubled, the number of comparisons increases approximately four times. This shows that the algorithm becomes less efficient as the input size becomes larger because the number of operations grows quadratically.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

INPUT SIZE	EXECUTION SPEED	SREENSHOT/S	OBSERVATION/S

1000	DIZOR:
	0 microseconds
	
	GRESOLA:
	0 microseconds
		DIZOR:
		`Time taken to search = 0 microseconds`
		`Student (727 562) does not need vaccination.`
		
		GRESOLA:
		`Time taken to search = 0 microseconds`
		`Student (703 192) needs vaccination.`
			The search was completed instantly, indicating efficient searching for a smaller data set.

10000	DIZOR:
	0 microseconds
	
	GRESOLA:
	0 microseconds
		DIZOR:
		`Time taken to search = 0 microseconds`
		`Student (727 562) needs vaccination.`
		GRESOLA:
		`Time taken to search = 0 microseconds`
		`Student (703 192) needs vaccination.`
			The search time remained unchanged as the number of records increased.

100000	DIZOR:
	1 microseconds
	
	GRESOLA:
	1 microseconds
		DIZOR:
		`Time taken to search = 0 microseconds`
		`Student (727 562) needs vaccination.`
		
		GRESOLA:
		`Time taken to search = 1 microseconds`
		`Student (703 192) needs vaccination.`
			The search time increased only slightly despite the significant increase in input size

ANALYSIS
DIZOR Based on the code, it implements a custom student recorder system and evaluates the performance of custom binary search algorithms across datasets. it measures only the execution window of 'needs_vaccination' based on your logged time, efficient metrics demonstrating the logarithmic scaling *GRESOLA* In this program, I noticed that even though the input size increased from 1,000 to 100,000, the runtime only changed slightly from 0 to 1 microsecond. This shows that the program can search through a much larger number of records without a significant increase in time, making binary search an efficient choice for handling larger data sets.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

GRESOLA

THEORETICAL ANALYSIS
Best Case: O(1) Worst Case: O(n^2)

EXPERIMENTAL ANALYSIS	
INPUT SIZE	**EXECUTION SPEED**
1000	Input Size: 1000 Result: Unique Execution Time: 2231 microseconds
10000	Input Size: 10000 Result: Unique Execution Time: 222817 microseconds
100000	Input Size: 100000 Result: Unique Execution Time: 21922168 microseconds

ANALYSIS AND COMPARISON
The Unique(A) algorithm has a theoretical operation count of T(n) = n^2 + 1, giving it a quadratic complexity of O(n^2). Experimentally, the execution time increased significantly as the input size increased. It took 2,231 microseconds for 1,000 elements, 222,817 microseconds for 10,000 elements, and 21,922,168 microseconds for 100,000 elements. This shows that the algorithm becomes much slower as the input size grows, which is consistent with its quadratic complexity.

INPUT

CPE010_HOA11P1 > suppleOne.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <chrono>

bool Unique(std::vector<int> A){

    int n = A.size();

    for (int i = 0; i <= n - 1; i++){
        for (int j = i + 1; j <= n - 1; j++){
            if (A[i] == A[j]){
                return false;
            }
        }
    }
    return true;
}

int main(){

    int sizes[] = {1000, 10000, 100000};

    for (int size : sizes){
        std::vector<int> A;
        for (int i = 0; i <= size - 1; i++){
            A.push_back(i);
        }

        auto start = std::chrono::steady_clock::now();
        bool result = Unique(A);
        auto end = std::chrono::steady_clock::now();
        auto time = std::chrono::duration_cast<std::chrono::microseconds>
                    (end - start).count();

        std::cout << "Input Size: " << size << std::endl;
        std::cout << "Result: "
                  << (result ? "Unique" : "Not Unique")
                  << std::endl;
        std::cout << "Execution Time: "
                  << time << " microseconds"
                  << std::endl;

        std::cout << "------------------------" << std::endl;
    }

    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

DIZOR:

THEORETICAL ANALYSIS
Best Case: O(1) **Worst Case: O(n^2)**

EXPERIMENTAL ANALYSIS

```
Input Size     Result         Execution Speed
-------------------------------------------------
1000           True           2 ms
10000          True           253 ms
100000         True           25174 ms

--------------------------------
Process exited after 26.73 seconds with return value 0
Press any key to continue . . .
```

INPUT SIZE	EXECUTION SPEED
1000	1000 True 2 ms
10000	10000 True 249 ms
100000	100000 True 28817 ms

ANALYSIS AND COMPARISON
Small discrepancy is visible at the smallest input n = 1000, where the runtime slightly exceeds the strict theoretical ratio relative to n = 10,0000. At lower input parameters, execution speeds are dominated by background thread switchings, console buffer setup, and standard vector memory allocation. As n approaches large scales, these constant setup overheads become fractions of the total processing timeframe that allows the loop execution behavior to dominate the timing profile

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <string>

using namespace std;

bool algorithmUnique(const vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

int main() {
    vector<int> inputSizes = {1000, 10000, 100000};

    cout << left << setw(15) << "Input Size"
         << setw(15) << "Result"
         << "Execution Speed" << endl;
    cout << string(50, '-') << endl;

    for (int n : inputSizes) {
        vector<int> worstCaseArray;
        worstCaseArray.reserve(n);
        for (int i = 0; i < n; i++) {
            worstCaseArray.push_back(i);
        }
```

```
35
36          auto startTime = chrono::high_resolution_clock::now();
37          bool isUnique = algorithmUnique(worstCaseArray);
38          auto endTime = chrono::high_resolution_clock::now();
39
40          auto durationMs = chrono::duration_cast<chrono::milliseconds>(endTime - startTime).count();
41
42          cout << left << setw(15) << n
43               << setw(15) << (isUnique ? "True" : "False")
44               << durationMs << " ms" << endl;
45      }
46
47      return 0;
48  }
49
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

GRESOLA:

THEORETICAL ANALYSIS	
SPACE COMPLEXITY: Best Case: LINEAR: T(n) = O(n) BINARY: T(n) = O(log n) Worst Case: LINEAR: T(n) = O(n) BINARY: T(n) = O(log n)	**TIME COMPLEXITY:** Best Case: LINEAR: T(n) = O(n) BINARY: T(n) = O(log n) Worst Case: LINEAR: T(n) = n + 1 → O(n) BINARY: T(n) = log_2n + 1 → O(log n)

EXPERIMENTAL ANALYSIS

```
x = 2, n = 10
rpower result: 1024
rpower time: 0 microseconds

brpower result: 1024
brpower time: 0 microseconds
```

ANALYSIS AND COMPARISON

The rpower algorithm has a linear complexity of O(n) because it decreases n by 1 in every recursive call. Meanwhile, brpower has a logarithmic complexity of O(log n) because it reduces n by approximately half in each recursive call. Both algorithms produced the correct result of 1024 when x = 2 and n = 10. Both recorded 0 microseconds, so there was no noticeable difference in their measured execution time for this small input. The theoretical analysis, however, shows that brpower is more efficient for larger values of n.

INPUT

suppleTwo.cpp > ...

```cpp
#include <iostream>
#include <chrono>

long long rpower(int x, int n){
    if (n == 0)
        return 1;
    return x * rpower(x, n - 1);
}


long long brpower(int x, int n) {
    if (n == 0)
        return 1;
    long long half = brpower(x, n / 2);

    if (n % 2 == 0)
        return half * half;
    return x * half * half;
}


int main(){

    int x = 2;
    int n = 10;

    auto start = std::chrono::steady_clock::now();

    long long result1 = rpower(x, n);

    auto end = std::chrono::steady_clock::now();

    auto time1 = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();

    start = std::chrono::steady_clock::now();

    long long result2 = brpower(x, n);

    end = std::chrono::steady_clock::now();

    auto time2 = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();


    std::cout << "x = " << x << ", n = " << n << std::endl;
    std::cout << "rpower result: " << result1 << std::endl;
    std::cout << "rpower time: " << time1 << " microseconds" << std::endl;
    std::cout << std::endl;


    std::cout << "brpower result: " << result2 << std::endl;
    std::cout << "brpower time: " << time2 << " microseconds" << std::endl;

    return 0;
}
```

DIZOR:

THEORETICAL ANALYSIS

Space complexity: **Linear:** **Best case: O(n)** **Worst case: O(n)** **Binary:** **Best case: O(log n)** **Worst case: O(log n)**	**Time complexity** **Linear:** **Best case: O(n)** **Worst case: O(n)** **Binary:** **Best case: O(log n)** **Worst case: O(log n)**

EXPERIMENTAL ANALYSIS

```
Input Size (n)    Linear Speed (rpower)    Binary Speed (brpower)
------------------------------------------------------------------
1000              46 ms                    1 ms
10000             486 ms                   0 ms
100000            4373 ms                  1 ms
------------------------------------------------------------------


--------------------------------
Process exited after 5.271 seconds with return value 0
Press any key to continue . . .
```

ANALYSIS AND COMPARISON

'Rpower' scales under a linear of O(n), while 'brpower' scales under an optimized logarithmic split path of O(logn). Because iteration presents timing metrics that approach hardware resolution. An accumulated benchmark of 10000 continuous repetition was utilized, resulting millisecond metrics reveal an alignment with the asymptotic curve.

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <string>

using namespace std;

double safeLinearPower(double x, int n) {
    double result = 1.0;
    for (int i = 0; i < n; i++) {
        result *= x;
    }
    return result;
}

double brpower(double x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        double y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        double y = brpower(x, n / 2);
        return y * y;
    }
}
```

```cpp
int main() {
    vector<int> inputSizes = {1000, 10000, 100000};
    double base = 1.00001;

    // We run each calculation 10,000 times to accumulate measurable time
    const int REPETITIONS = 10000;

    cout << left << setw(18) << "Input Size (n)"
         << setw(25) << "Linear Speed (rpower)"
         << "Binary Speed (brpower)" << endl;
    cout << string(65, '-') << endl;

    for (int n : inputSizes) {

        // 1. Measure Linear Speed over 10,000 runs
        auto start1 = chrono::high_resolution_clock::now();
        for (int r = 0; r < REPETITIONS; r++) {
            safeLinearPower(base, n);
        }
        auto end1 = chrono::high_resolution_clock::now();
        auto durationLinear = chrono::duration_cast<chrono::milliseconds>(end1 - start1).count();

        // 2. Measure Binary Speed over 10,000 runs
        auto start2 = chrono::high_resolution_clock::now();
        for (int r = 0; r < REPETITIONS; r++) {
            brpower(base, n);
        }
        auto end2 = chrono::high_resolution_clock::now();
        auto durationBinary = chrono::duration_cast<chrono::milliseconds>(end2 - start2).count();

        // Format outputs into clear, stable millisecond values
        string linearStr = to_string(durationLinear) + " ms";
        string binaryStr = to_string(durationBinary) + " ms";

        cout << left << setw(18) << n
             << setw(25) << linearStr
             << binaryStr << endl;
    }

    cout << string(65, '-') << endl;
    return 0;
}
```

CONCLUSION/LEARNING

DIZOR

Measuring algorithmic time complexities required to balance both theoretical and experimental analysis. Theoretical analysis acts as a mathematical blueprint that predicts how an algorithm scales as input sizes. Experimental acts as a validation tool that reveals the constraints, memory hierarchies, and compiler optimizations affect actual execution speeds

GRESOLA

Overall, this supplementary activity helped me understand how algorithm complexity can be analyzed theoretically and experimentally. I learned that by observing the structure of a given code, such as its loops and recursive calls, I can theoretically determine how its performance changes as the input size increases. I was then able to support these observations experimentally by implementing the algorithms and measuring their actual execution times. This activity showed me that theoretical and experimental analysis should be balanced when measuring algorithmic complexity: theoretical analysis provides a basis for predicting an algorithm's efficiency, while experimental analysis provides actual evidence of how the algorithm performs in practice. Through this, I realized that analyzing an algorithm is not only about determining whether it works, but also about understanding why it performs the way it does and how efficiently it handles larger inputs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT and Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: We used ChatGPT and Gemini for code assistance and ChatGPT for grammar correction. These tools were used only to support our understanding and improve our work. It did not replace our original thinking, analysis, authorship, or completion of the required tasks.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.